

ООП в Python

заметки по видео-урокам

Мурашев Илья

September 24, 2026

1 Введение

1.1 Объекты(данные и поведение)

- `type(obj)` – Выводит тип объекта. Каждый объект лежит в каком-либо классе.

```
1 a = 3
2 print(type(a))
3 << < class 'int' >
```

- `isinstance(obj, type)` – проверяет соответствие объекта `obj` переданному классу `type`, возвращает `bool`.

```
1 a = 3
2 print(isinstance(a, int))
3 << True
```

Заметка

В качестве типа(второго аргумента) можно передать параметр `object`, а в качестве `obj`(первого аргумента) любой тип. Тогда функция будет возвращать `True` на любом типе, что доказывает, что любой тип в Python является объектом.

1.2 Классы

Определение

Класс – шаблон, при помощи которого можно создавать объекты. В классе определяются и описываются объекты и их поведение.

```
1 class Car:
2     pass
```

Listing 1: пример создания пустого класса

```
1 class Car:
2     pass
3 a = Car()
4 print(type(a))
5 << __main__.Car
```

Listing 2: Пример создания экземпляра класса Car

В классе можно определить поля(по умолчанию они публичные):

```
1 class Car:
2     model = "BMW"
3     engine = 1.6
```

1.3 Атрибуты класса

```
1 class Person:
2     name = 'Ivan'
3     age = 30
```

Определение

Поля `name` и `age` называются атрибутами класса.

- `ClassName.__dict__` – позволяет выводить всю информацию(в том числе об атрибутах) о классе.

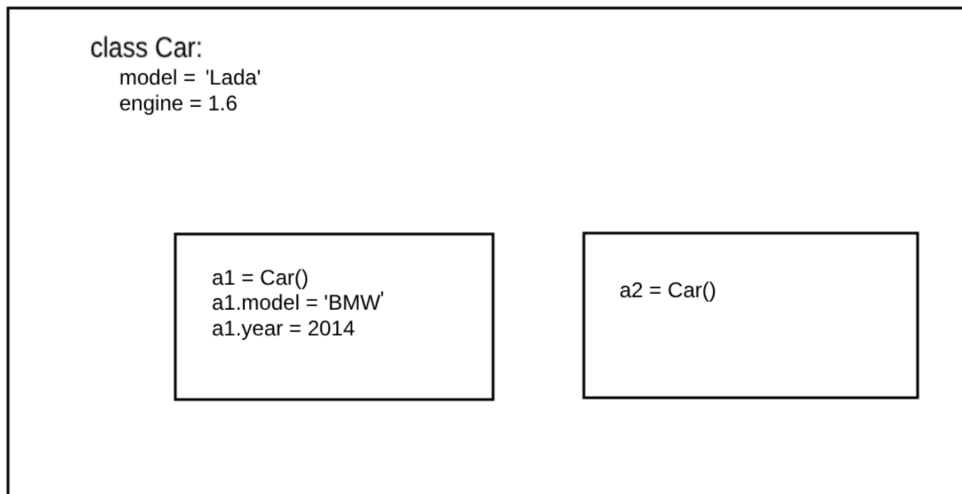
```
1 class Car:
2     model = "LADA"
3     engine = 1.6
4
5 print(Car.__dict__)
6 << {'__module__': '__main__', '__firstlineno__': 1, 'model': 'LADA',
    'engine': 1.6, '__static_attributes__': (), '__dict__': <attribute
    '__dict__' of 'Car' objects>, '__weakref__': <attribute
    '__weakref__' of 'Car' objects>, '__doc__': None}
```

- `getattr(Car, "model", 200)` – (название класса, название поля в кавычках, код ошибки) – безопасное обращение к полю: возвращает его значение или значение ошибки если поля нет. Можно прокидывать как класс, так и экземпляр класса.
- `setattr(ClassName, 'AttributeName', value)` – позволяет добавлять поле в созданный ранее класс.
- `ClassName.NewAttribute = value` – такой синтаксис создания атрибута класса тоже работает. похоже на объявление новой переменной.
- `delattr(ClassName, 'AttributeName')` – позволяет удалять поле внутри в созданного ранее класса.
- `del ClassName.AttributeName` – аналогичный синтаксис удаления поля класса
- `hasattr(ClassName, AttributeName)` – проверяет, есть ли у переданного класса заданный атрибут, возвращает `bool`.

Заметка

`setattr`, `delattr` при вызове объекта самого класса изменяют значения у всех экземпляров класса. Их можно вызывать для самих экземпляров, тогда изменения будут локальны.

1.4 Атрибуты экземпляра класса



В примере есть класс `Car` и его два экземпляра `a1` и `a2`. `a1` наследует все поля родителя. Может их изменять и создавать новые локально в своей области видимости. `a2` пуст и все поля, которые есть у него – это копии полей родителя.

1.5 Функция как экземпляр класса

Внутри класса(не экземпляра класса) можно объявлять как поля, так и методы(функции).

```

1 class Car:
2     model = 'BMW'
3     engine = 1.6
4
5     def drive():
6         print("Let's go!")
7
8 Car.drive() # <-- reference to class name, not to variable
9 << Let's go!
  
```

Заметка

При таком написании вывод функции можно увидеть только если обратиться именно к самому классу `Car` и к **функции** внутри него. Если создать экземпляр класса `b = Car()` и вызвать уже **метод** `b.drive()` компилятор вернет ошибку: `TypeError: Car.drive() takes 0 positional arguments but 1 was given`. Данная реализация функции не принята!

Для лечения такой вещи можно воспользоваться декоратором `@staticmethod`:

```

1 class Car:
2     model = 'BMW'
3     engine = 1.6
4
5     @staticmethod
6     def drive():
7         print("Lets go!")
8
9 Car.drive() # <-- reference to class name
10 << Lets go!
11 b = Car()
  
```

```

12     b.drive()      # <-- reference to class variable name
13     << Lets go!

```

1.6 Методы класса. Аргумент self.

Заметим интересную деталь:

```

1     class Cat:
2     def hello():
3         print('Hello, everyone!')
4
5     Bob = Cat()
6     print(Cat.hello)
7     print(Bob.hello)
8
9     << <function Cat.hello at 0x70e2a839f270>
10    << <bound method Cat.hello of <__main__.Cat object at 0x70e2a8390d70>>

```

Главные отличия понятия **функции** класса и **метода** класса:

- Метод – та же функция, только объявленная внутри класса.
- Метод привязан к конкретному объекту(объект Bob в примере). Функция привязана целиком к классу и вызывается от него(класс Cat).
- При вызове метода, тот объект, который с ним связан, автоматически прокидывается внутрь метода в качестве аргумента.

```

1     class Cat:
2     def hello(*args):
3         # = f(...), get random
4         # amount of arguments
5         print('Hello, everyone!', args) # output given arguments
6
7     Bob = Cat()
8     print(Cat.hello())
9     print(Bob.hello())
10
11    << Hello, everyone! ()
12    << Hello, everyone! (<__main__.Cat object at 0x7c5144730ec0>,)

```

Вывелся адрес хранения объекта Bob в памяти.

Заметка

При вызове метода обязательно(согласно синтаксису) внутрь передается 1 аргумент – сам объект класса. (*this прокидывается всегда и именуется первым аргументом). Его обычно называют `self`, однако это лишь условно и название этого параметра может быть произвольно.

```

1 class Cat:
2     breed = 'pers'
3     age = 5
4     name = 'default'
5     def SetCat(self, age, name):
6         self.age = age
7         self.name = name
8     def Hello(self):
9         print(f"Hello, {self.name}!")
10
11 Bob = Cat()
12 Bob.SetCat(6, 'Bob')
13 Bob.Hello()
14
15 << Hello, Bob!

```

Создали класс `Cat` с атрибутами `breed`, `age`, `name`. После этого создали метод-конструктор, который присвоил указанным полям переданные значения. Обращается он к экземпляру класса – к `self` (аналог `*this`).

Заметка

метод `SetCat` нельзя вызывать без аргументов `age`, `name`. Иначе это приведет к ошибке. Можно использовать = default значения параметров:
`def SetCat(self, age = 5, name = 'Stiv') :`

1.7 Инициализация объекта. Метод `init`.**Определение**

Метод `__init__` – специальный метод классов, предназначенный для инициализации полей объекта(= конструктор). Вызывается сразу после создания экземпляра класса.

```

1 class Cat:
2     breed = 'pers'
3     age = 5
4     name = 'default'
5     def __init__(self, breed, age, name):    #constructor
6         self.breed = breed
7         self.age = age
8         self.name = name
9     def Hello(self):
10        print(f"Hello, {self.name} {self.breed} {self.age}!")
11 Bob = Cat('Russian', 6, 'Ivan')           #construct specific object Bob
12 Bob.Hello()                               #call method
13 << Hello, Ivan Russian 6!

```

Теперь метод `SetCat` полностью убрался в конструктор `__init__`

Заметка

Обычно, аналогично с++, имена переменных в конструкторе называют аналогично соответствующим инициализации параметрам.

В качестве примера создадим класс точки. Добавим конструктор и методы: напечатать точку, вывести расстояние до другой точки, и переместиться в другую точку.

```

1 class Point:
2     x = 0
3     y = 0
4
5     def move_to(self, new_x, new_y):
6         self.x = new_x
7         self.y = new_y
8
9     def __init__(self, coord_x = 0, coord_y = 0): #(0,0) = default
10        self.move_to(coord_x, coord_y)
11
12    def go_home(self):
13        self.move_to(0,0)
14
15    def print_coord(self):
16        print(f"Точка скоординатами ({self.x},{self.y})")
17
18    def dist(self, point):
19        if(not isinstance(point, Point)):
20            raise ValueError("Аргумент должен принадлежать классу Point")
21        return ((self.x - point.x)**2 + (self.y - point.y)**2)**0.5

```

1.8 Моносостояния

В Python нет ссылок как в C++. Однако, например, для словарей по умолчанию операция `=` означает привязка обоих имен к одному объекту в памяти (аналог ссылки), а не копирование одного в другой. В силу этого можно реализовать следующую структуру: у класса `Car` общий словарь `__shared_attr` с полями `'brand'` и `'volume'`. При создании в `__init__` каждый экземпляр класса состоит из буквально этого же словаря. Поэтому изменения полей в одном влекут изменения в другом.

```

1 class Car:
2     __shared_attr = {
3         'brand': 'Lada',
4         'volume': 3
5     }
6
7     def __init__(self):
8         self.__dict__ = Car.__shared_attr
9
10    def info(self):
11        print(f"Машина марки {self.__shared_attr['brand']} и размера {self.__shared_attr['volume']}")
12
13 b = Car()
14 b._Car__shared_attr['volume'] = 5
15 c = Car()
16 c.info()
17
18 << Машина | марки | Lada и | размера | 5

```

1.9 Модификаторы доступа

В классах в Python существуют модификаторы доступа полей и методов:

- `public` - поля. По умолчанию все поля и методы в статусе `public`. Доступ к ним открыт вне класса.
- `protected` - поля. Просто договоренность между программистами, что отмеченная переменная или метод используются только для внутреннего устройства класса. Доступ вне класса такой же как у `public`. Для создания `protected` переменной или метода нужно поставить `_` перед названием.
- `private` - поля. поля и методы, защищенные от внешнего доступа. Для создания `private` переменной или метода нужно поставить `__` перед названием.

```
1 class Car:
2     def __init__(self, engine, wheel, price):
3         self.engine = engine
4         self._wheel = wheel
5         self.__price = price
6
7 b = Car(1.6, 35, 1000)
8 print(b.engine)
9 print(b._wheel)
10 print(b.__price)
11 << 1.6
12 << 35
13 << Traceback (most recent call last):
14   File "/home/daydreamer/snap/pyp_prog/test_2.py", line 10, in <module>
15     print(b.__height)
```

Определение

Инкапсуляция – важное свойство ООП, позволяющее скрывать внутреннее устройство классов и методов извне класса(существование `private` и `public` полей).

1.10 Property. Getter and setter

Для реализации идеи инкапсуляции так же используется специальный метод `property`. С его помощью можно использовать логику доступа, создания и удаления private переменных извне класса. Обычно он может включать в себя какую-то нетривиальную логику по созданию, выводу и удалению полей класса.

```
1 class BankAccount:
2     def __init__(self, name, balance):
3         self.__name = name          #private - var
4         self.__balance = balance    #private - var
5
6     def get_balance(self):
7         return self.__balance
8
9     def set_balance(self, new_balance):
10        if not isinstance(new_balance, (int, float)):
11            raise ValueError('Баланс должен быть числом')
12        self.__balance = new_balance
13    def del_balance(self):
14        del self.__balance
15
16        balance = property(fget=get_balance, fset=set_balance, fdel=del_balance)
17
18 q = BankAccount("Ivan", 200)
19 print(q.balance)                # access to get private balance
20 q.balance = 300                  # access to set private balance
21 print(q.balance)
22 del q.balance                    # access to del private balance
23 print(q.__dict__)
24 q.balance = 300
25 print(q.__dict__)
26 << 200
27 << 300
28 << {'_BankAccount__name': 'Ivan'}
29 << {'_BankAccount__name': 'Ivan', '_BankAccount__balance': 300}
```

Пример использования метода `property`